WE CODE CODERS · ROUND 3 · FINAL PRODUCT

Baton

Evidence-backed project handovers that stay true

When someone leaves a project, Baton turns its scattered files, chats and GitHub activity into a **checked, approved and permission-safe handover**. Every line links to a real quote, gaps become questions for the person leaving, the new member signs in to see only what they may see, asks questions answered from evidence, and the handover **updates itself as the project moves on**.

Team	Team Pixels
Institution	JSPM's Rajarshi Shahu College of Engineering (RSCOE), Pune — CSBS, Third Year
Members	Shravani Yalagatti (Team Lead) · Yash Solanke · Atharva Shete
Problem statement	Prevent Critical Knowledge From Leaving With People
Live app	https://baton-weld.vercel.app
Source code	github.com/shravanisy2006/Baton
Status	**All three improvements committed in Round 2 are delivered**, tested and live

Contents

1. Baton in one minute, and how to try it
2. The three improvements we committed to — delivered
3. Complete list of features
4. The main journey, end to end
5. Technical architecture and implementation
6. Reliability under normal use
7. Testing details
8. Usability and real-world fit
9. Product roadmap and future development
10. Known limits, team and disclosures

1. Baton in one minute, and how to try it

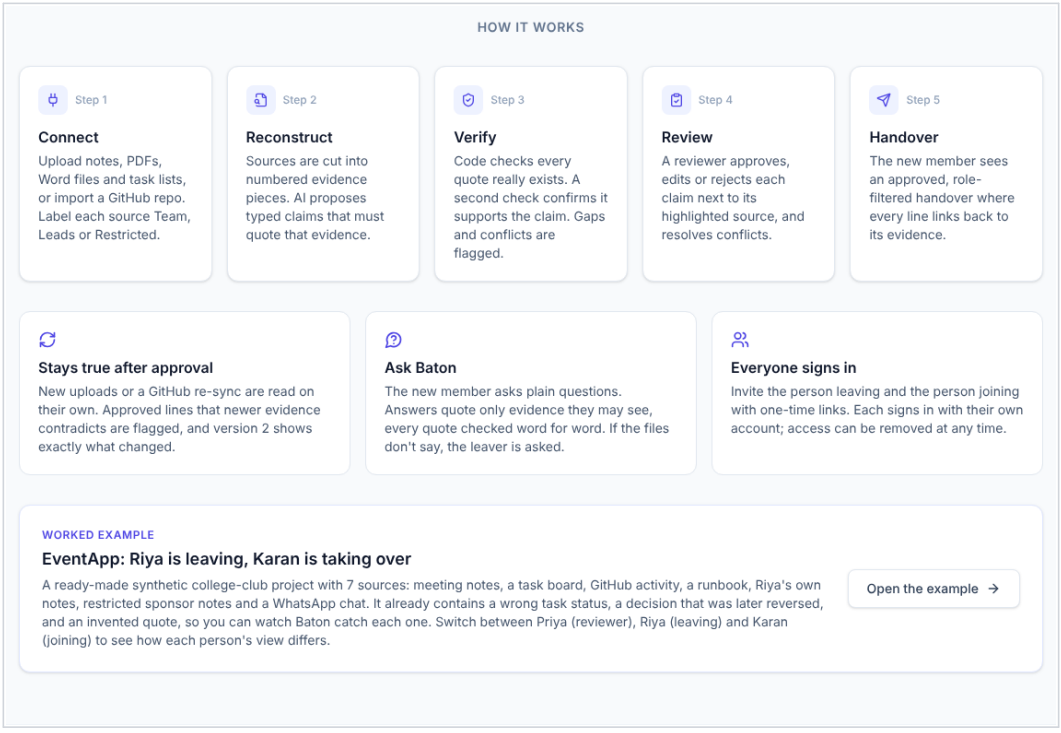
Most AI tools read everything and write a confident summary; you can't tell what is true, outdated or invented. Baton works the other way round: **AI proposes, code checks, people decide** — and when the project changes, Baton re-checks only what changed and shows the difference.

> Think of a careful teacher checking answers against the textbook. The AI is a quick student that sometimes makes things up; the project's own files are the textbook. A fact is accepted only if the AI points to the exact words in the textbook and a program confirms they are really there.

Two ways a judge can use the live link, with no help from us

Path	What to do
A. Your own project (about 10 min)	Try it with your project → create an account → name who is leaving and who is taking over → upload files (or the downloadable sample project) → Run analysis → settle conflicts → review → generate and approve the handover → **Invite** the new member and open the link in a private window with a second account → read the handover and **Ask Baton** → back as reviewer, upload a newer file → **Analyse new evidence** → see **Changed since approval** → make **version 2** and see what changed.
B. Worked example (about 3 min, no account)	See how it works → Open the example. EventApp (synthetic): Riya is leaving, Karan is taking over, Priya reviews. Planted problems — a stale task status, a reversed decision, an invented quote, an altered quote, a restricted contact — show Baton catching each one. Switch people with “Viewing as”.

A pop-up guide points at the next button to click for each person at each stage, so the journey can be completed without a walkthrough.



Home page: the three Round 3 capabilities, and the worked example under “How it works”.

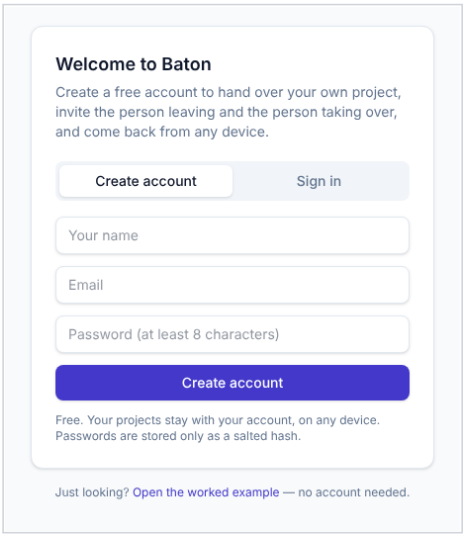
2. The three improvements we committed to — delivered

Round 3 is measured against the three improvements named in our Round 2 submission. All three are built, tested and live. Nothing was swapped for something easier.

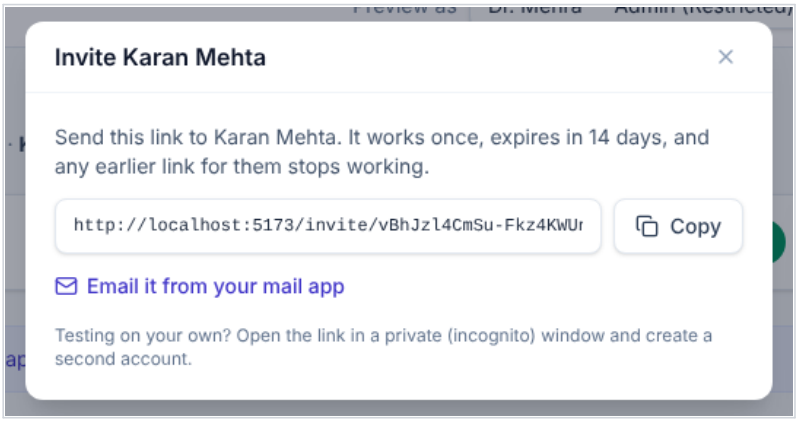
Committed in Round 2	Delivered in Round 3	Proved by
1. Living handover — re-sync after approval, flag changed lines, reviewed version 2 with a diff, re-acknowledge only changed sections	**Yes.** Update runs read only new evidence and keep every decision; GitHub re-sync adds only new/changed items; “Changed since approval” flags; “What changed since version N” diff; acknowledgements carried over for unchanged sections	test_living_handover…, test_github_resync…; browser journey
2. Ask Baton — answers only with checked quotes; unanswerable questions go to the leaver; answers become evidence	**Yes.** Access-filtered retrieval, cited answers with highlights, refusal when unsupported, “Ask the leaver instead”, the answer is cited for the next person	test_ask_baton…, test_ask_unanswered…; browser journey
3. Real team accounts with invite links — own sign-in per person, per-account roles and access, real names in the audit log, revocable access	**Yes.** Sign up / sign in, one-time invite links, join with your own account, remove access, previews of not-yet-joined people logged as previews	test_signup…, test_invite_accept_preview_and_remove; second-browser journey

2.1 Real team accounts with invite links

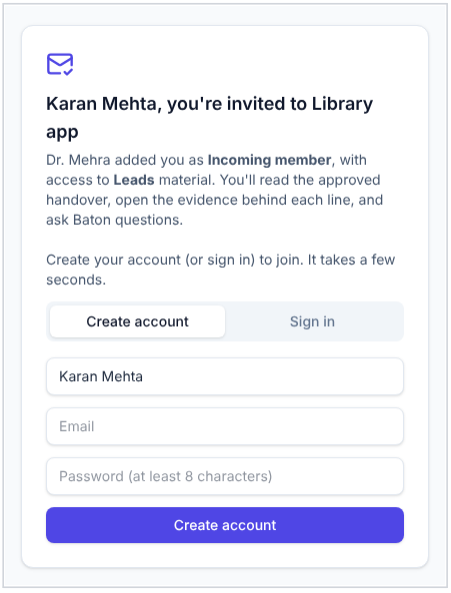
The reviewer creates an account and the project, naming the person leaving and the person taking over. Each gets a **one-time invite link** (valid 14 days, stored only as a hash; a new link cancels the old one; any pending link can be cancelled). The person opens it, sees exactly what they are joining, creates their own account and presses Join: their place in the project moves to their account. From then on they sign in from any device, nobody can preview as them, and the audit log shows their real name. An admin can remove anyone's access immediately.



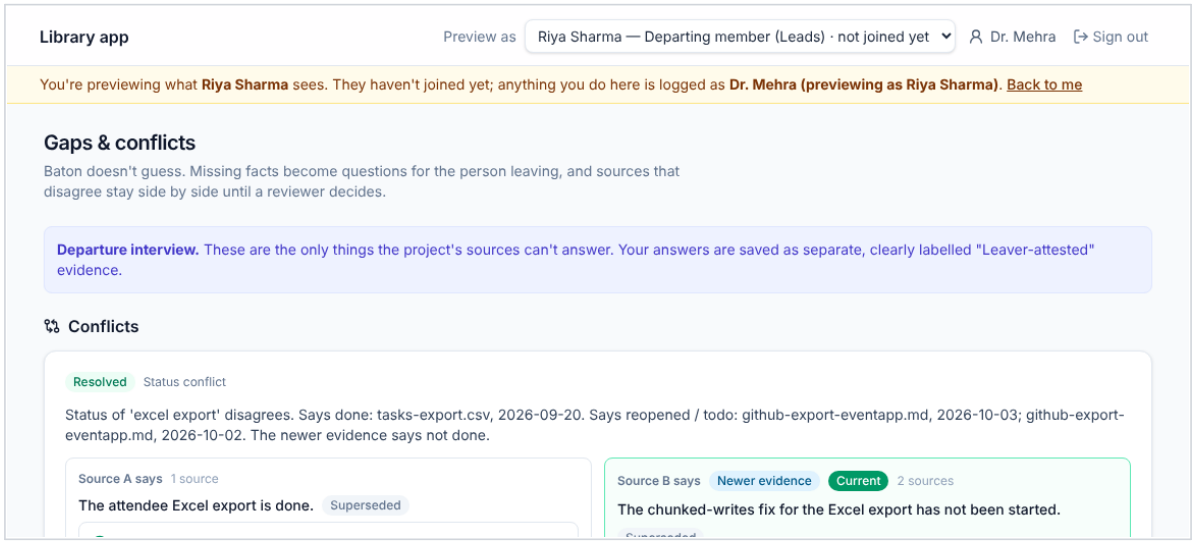
Sign up / sign in. Passwords are stored only as salted scrypt hashes; repeated failed sign-ins are blocked for 15 minutes.



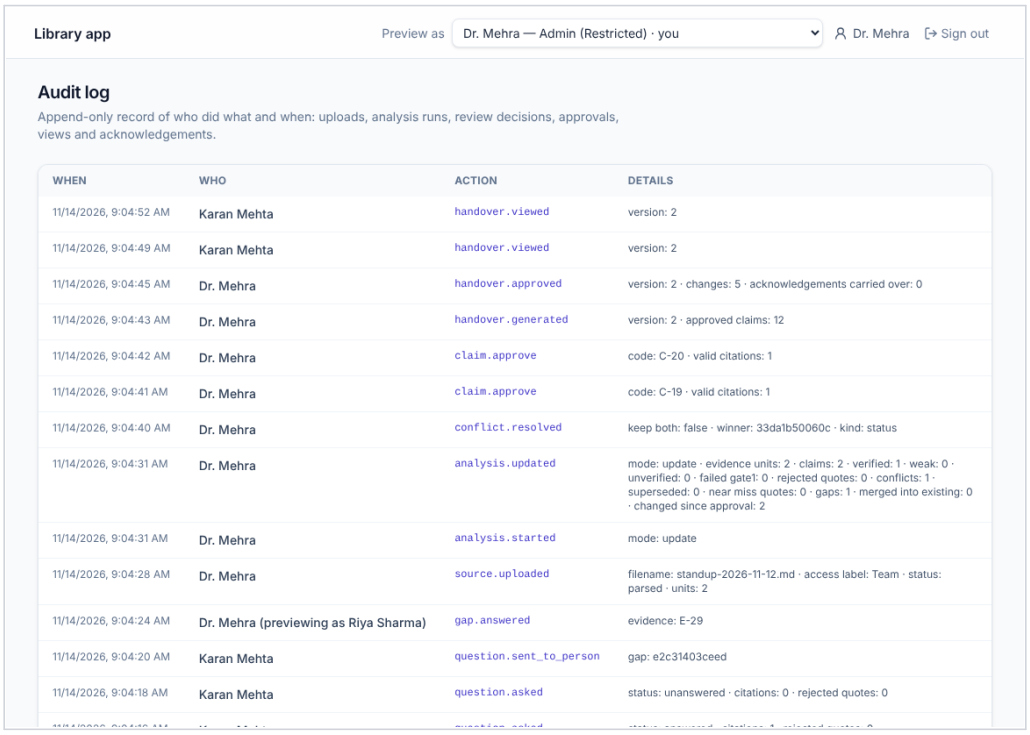
Invite: a one-time link, copied or emailed from the reviewer's own mail app.



What Karan sees when he opens the link: the project, his role and access, then sign up and Join.



Before someone joins, the reviewer can preview their view. A banner says so, and everything done this way is logged as “Dr. Mehra (previewing as Riya Sharma)”.



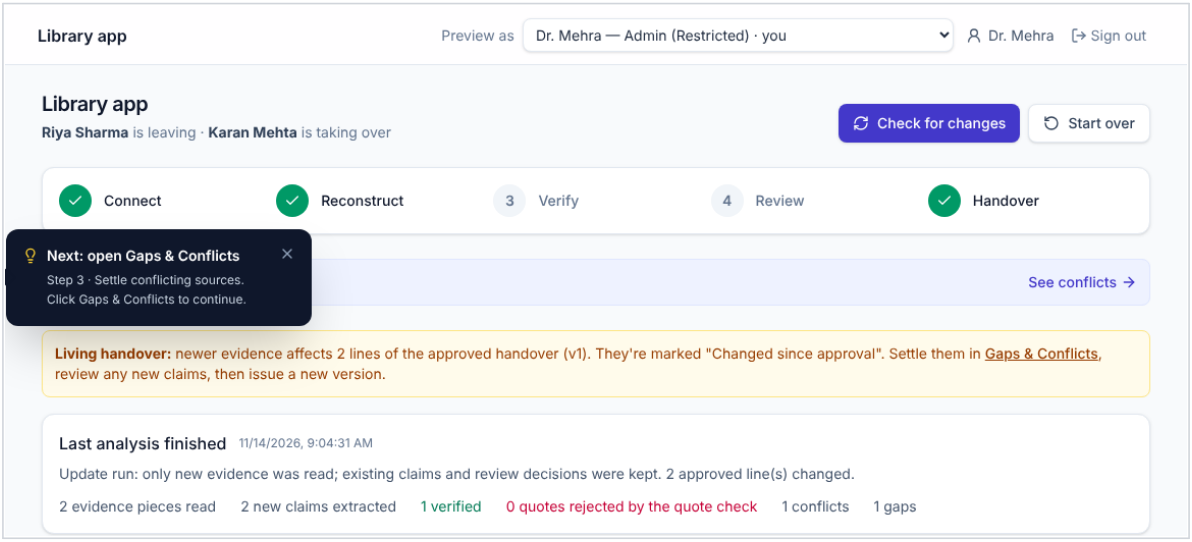
Audit log after the journey: real names for Karan's own actions, “previewing as” for the reviewer's, and the update run, version 2 approval and carried-over acknowledgements.

2.2 Living handover

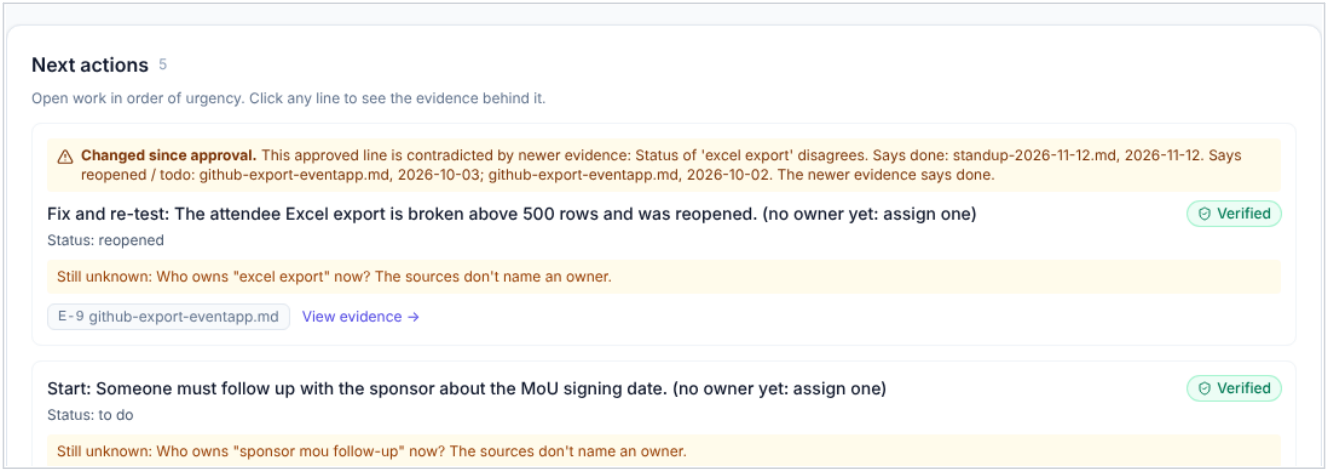
A handover is only useful while it is true. After approval, the reviewer uploads newer files or presses **Check for changes** (which re-syncs GitHub). Baton runs an **update run**: only evidence that hasn't been analysed is read, through the same extraction and quote checks. Every earlier review decision stays. New claims are compared with existing ones:



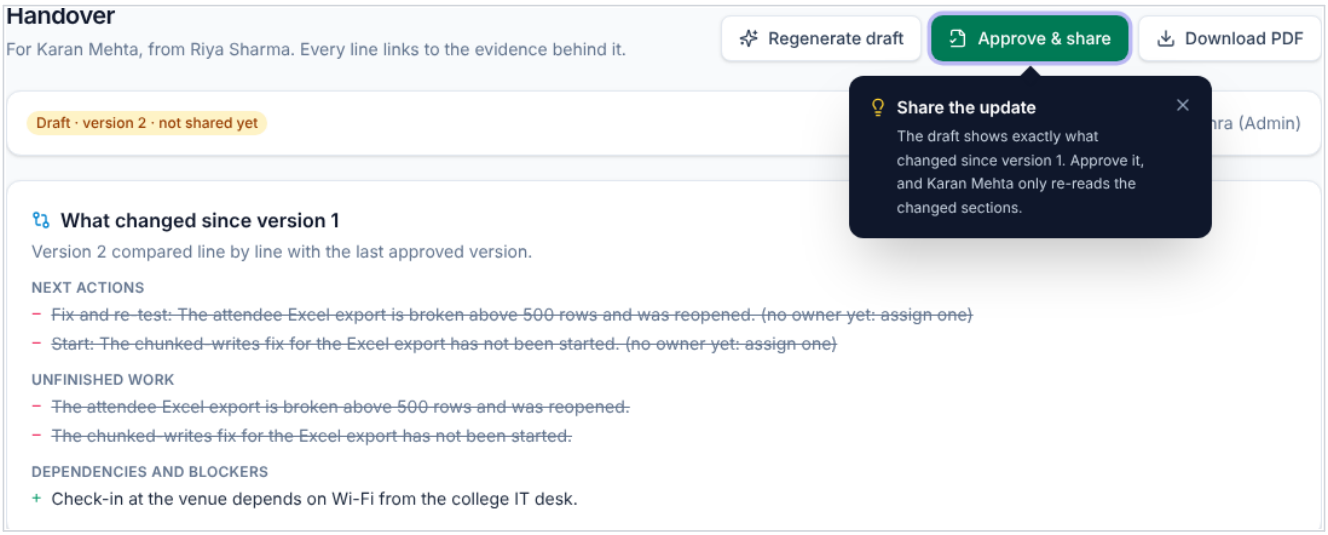
- A restatement of an existing claim adds its quote as extra support (no duplicate line).
- A claim that contradicts or replaces an **approved** one raises a conflict and marks that approved line **Changed since approval** for everyone — including the new member reading the current version.
- The reviewer settles it; Baton then knows the approved handover is out of date and offers **Make version 2**. The draft carries a line-by-line **What changed since version 1** panel (added, updated, removed), filtered by each reader's access level. When it is approved, the new member keeps their acknowledgements for unchanged sections and re-reads only the changed ones.



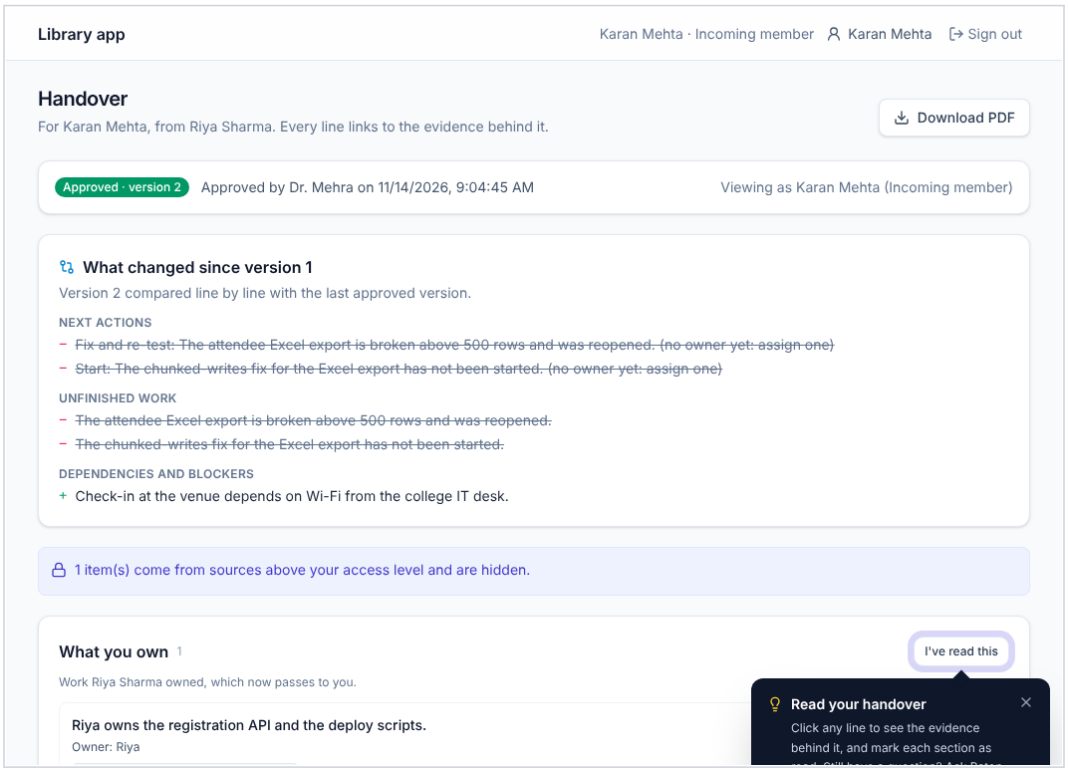
After uploading a newer standup note: “Analyse 2 new pieces” ran as an update — existing claims and decisions kept, 2 approved lines changed.



The approved handover (version 1) now flags the Excel line: newer evidence says the fix is done and deployed.



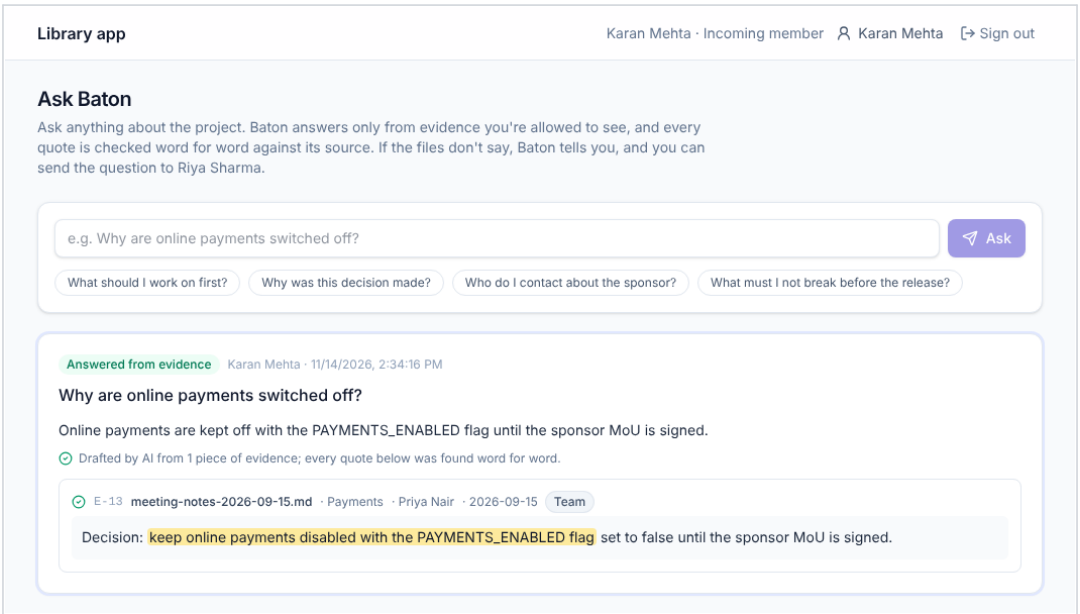
Version 2 draft: what changed since version 1. The finished Excel work leaves the handover; a new dependency arrives.



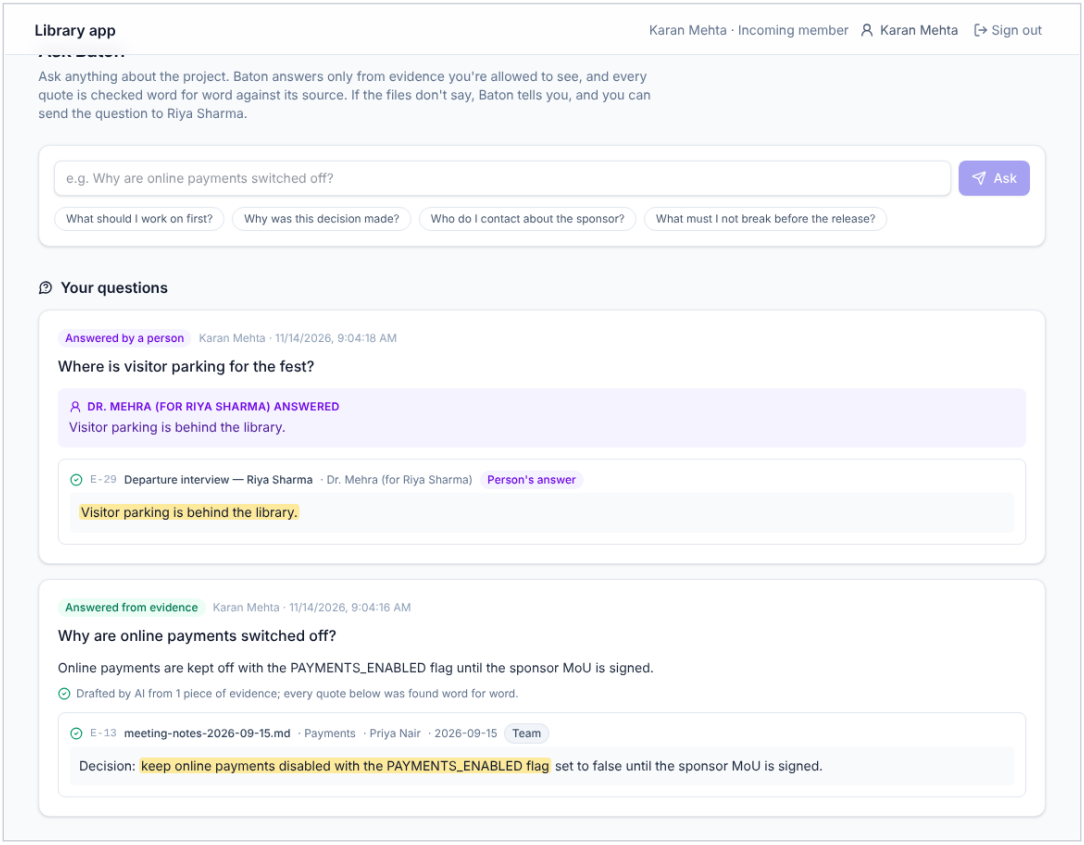
Karan, signed in with his own account, sees version 2 and the same diff. Only changed sections need reading again.

2.3 Ask Baton

The new member asks plain questions. Retrieval runs in code over evidence **at or below the asker's access level** (including earlier answers from the leaver), so the AI never sees anything the asker can't. Gemini drafts a short answer and must cite evidence IDs with exact quotes; every quote goes through the same word-for-word check, and failing quotes are dropped. If no quote survives, Baton says the files don't answer this — it doesn't guess. One click sends the question to the leaver (or a reviewer if the leaver has gone); the answer is saved as labelled evidence and cited the next time anyone asks.



An answer from evidence, with the quote highlighted in its source, author, date and access label.



A question the files couldn't answer went to Riya; her answer (entered here by the reviewer while previewing, and labelled so) now comes back as cited evidence.

One change of method, explained

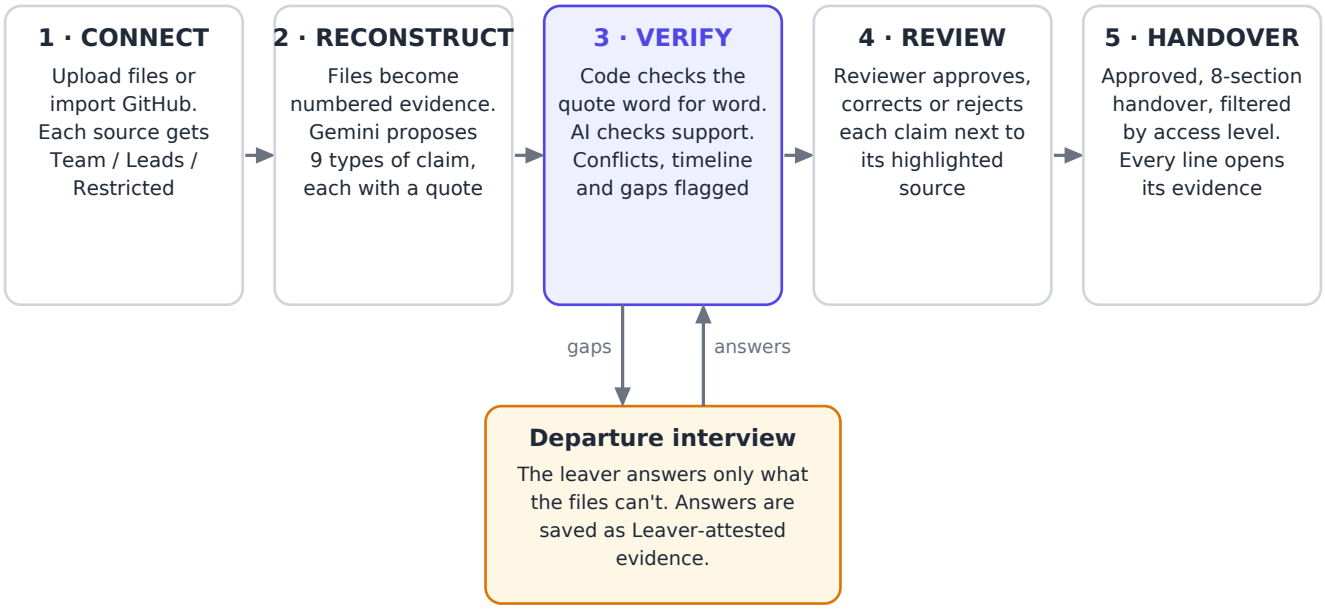
We committed to Supabase Auth. We built Baton's own email-and-password accounts in the same API instead (scrypt hashes, hashed one-time invite tokens). Supabase's built-in email sender allows only a few messages per hour on the free plan and needs a separate SMTP service for real use, and a second auth system would have split permission checks across two places. The commitment — each person signs in to their own account, invited by link, with revocable access and real names in the log — is met in full. Invite links are copied or emailed from the reviewer's own mail app; automatic email delivery is first on the roadmap.

3. Complete list of features

Everything below works on the live link.

Area	Features
Accounts and team	Sign up / sign in (scrypt, sign-in throttling) · projects that name the leaver and the new member with access levels · one-time invite links (14 days, hashed, revocable, replaced on re-issue) · join with your own account · preview not-yet-joined people (bannered and logged) · remove access · roles Admin, Reviewer, Departing, Incoming, Viewer
Connect	PDF, DOCX, TXT, Markdown, CSV, JSON (GitHub / Slack / Discord / any records), WhatsApp exports · read-only GitHub import and re-sync (only new/changed items) · Team / Leads / Restricted labels · secret and phone redaction · coverage report · downloadable sample project
Reconstruct	Numbered evidence units with source, author, date/time, page/row/section/message · Gemini extraction of 9 claim types as validated JSON · retry then reject · backup model · response cache · 90 s timeout
Verify	Gate 1 word-for-word quote check in code with highlight offsets · Gate 2 AI support check · 7 confidence labels · status/owner conflicts one per topic with the newer side marked · timeline check for reversed decisions · gaps: missing owner/reason, open questions, unreadable sources · departure interview answers as labelled evidence
Review	Approve, approve with a reason (Reviewer-asserted), reject, correct, reassign owner, undo · conflicts block approval · settling a conflict supersedes the losing side
Handover	8 sections incl. Next actions · built only from approved claims · draft → approved versions · evidence drawer on every line · per-section acknowledgement · PDF download · access re-checked on every read; hidden items counted
Living handover	Update runs on new evidence only, decisions kept · Check for changes · Changed-since-approval flags for every reader · out-of-date detection · version N+1 with access-filtered diff · acknowledgement carry-over · Start over keeps the approved handover visible
Ask Baton	Access-filtered retrieval · cited answers with highlighted, checked quotes · refusal when unsupported · Ask the leaver instead (or a reviewer) · answers become cited evidence · question history
Records and usability	Append-only audit log of every upload, run, decision, invite, join, removal, approval, view, acknowledgement and question · pop-up step guide per role · worked example · phone layout · clear errors with a way out

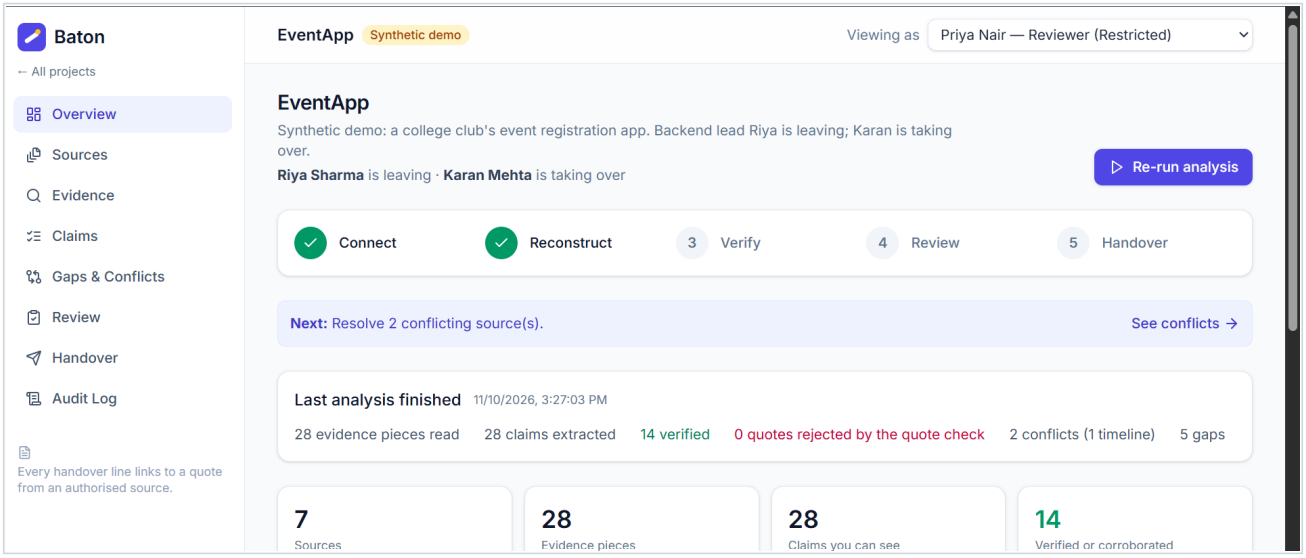
4. The main journey, end to end



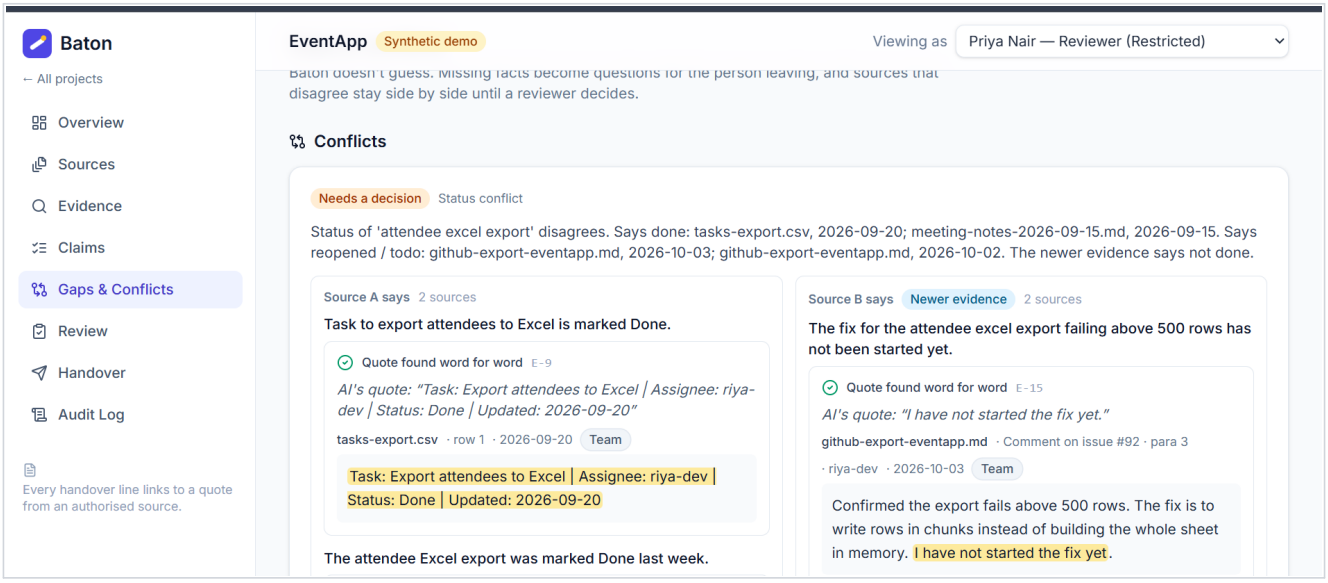
Nothing reaches the new member without a checked quote, a human approval and a permission check.

Step	Who	What happens
Set up	Reviewer	Creates an account and a project naming the leaver and the new member; invites both
Connect	Reviewer	Uploads files / imports GitHub with access labels; Baton parses, redacts and numbers the evidence
Reconstruct + Verify	Baton	Extracts claims with exact quotes; checks every quote; flags conflicts, reversed decisions and gaps
Interview	Leaver	Signs in from the invite and answers only what the files can't; answers become labelled evidence
Review	Reviewer	Settles conflicts; approves, corrects or rejects each claim next to its highlighted source
Handover	Reviewer → New member	Approved, access-filtered handover; the new member opens evidence, acknowledges sections, asks Baton
Keep it true	Reviewer → New member	New evidence → update run → changed lines flagged → version N+1 with a diff → re-read only changes

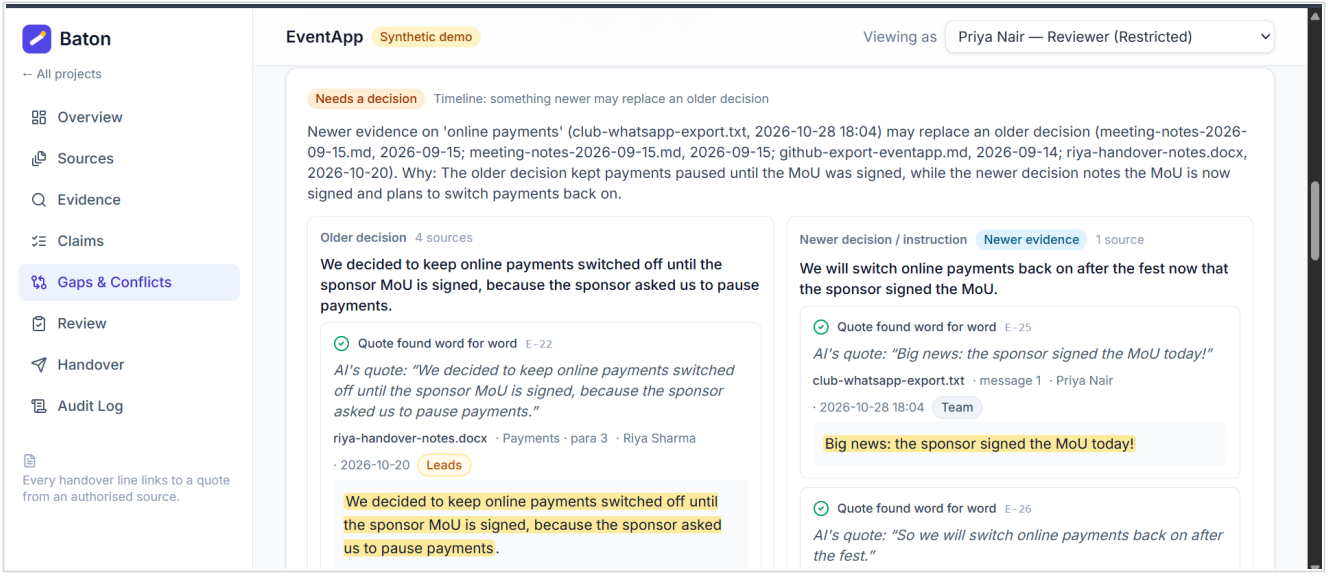
The same pipeline on the worked example with real Gemini (saved answers)



Overview after a real Gemini run: 28 evidence pieces, 28 claims, 14 verified, 2 conflicts (1 timeline), 5 gaps.

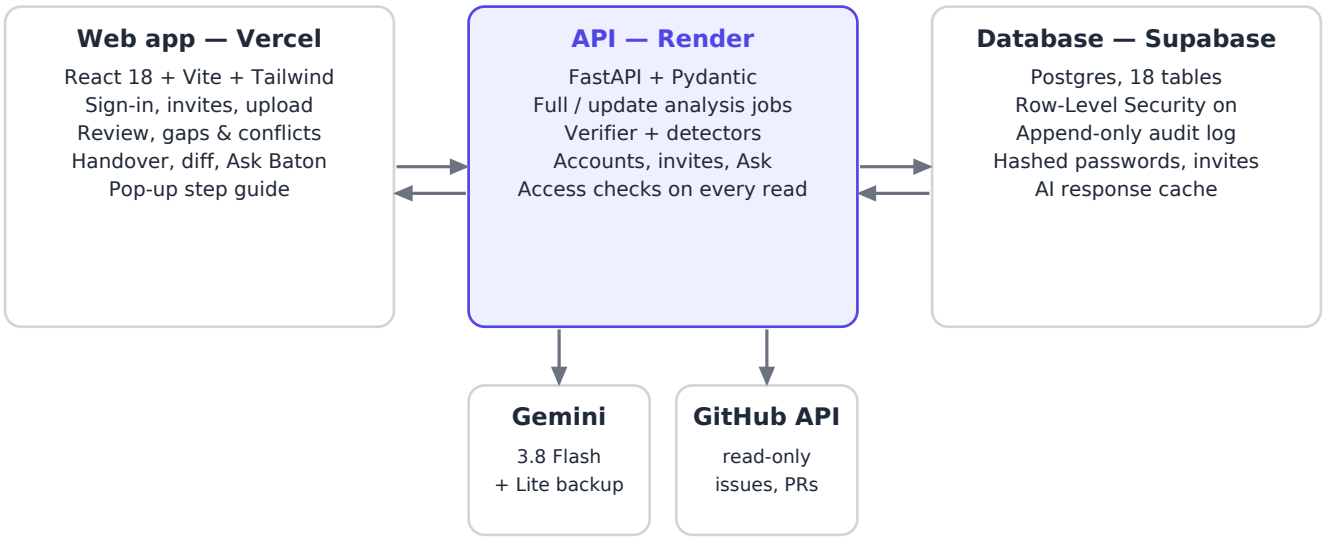


Status conflict: two older sources say the Excel export is done; two newer GitHub comments say it was reopened.



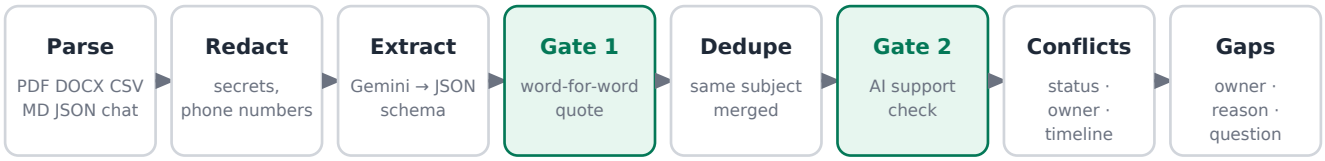
Timeline check: "keep payments off until the MoU" (four older sources) vs "MoU signed, payments back on" (WhatsApp, newer).

5. Technical architecture and implementation



Layer	Technology	Why
Frontend	React 18, TypeScript, Vite, Tailwind CSS, React Router	Clear review screens; free hosting on Vercel
Backend	Python, FastAPI, Pydantic 2, SQLAlchemy 2.1, Uvicorn	Best document-parsing and AI libraries; every AI response validated
Database	Supabase Postgres (psycopg 3, Session pooler); SQLite locally	Managed Postgres with Row-Level Security; same code on a laptop
AI	Gemini 3.8 Flash + Gemini 3.5 Flash-Lite backup (google-genai, JSON schema)	Structured output; free tier; automatic fallback
Parsing / matching	PyMuPDF, python-docx, pandas, custom chat/JSON parsers, RapidFuzz	Keeps page/row/message locations for citations
Auth	PyJWT sessions, scrypt hashes (standard library), SHA-256-hashed invite tokens	No secrets stored in plain text
Integration	GitHub REST API (read-only), httpx	Official, authorised access
Hosting	Vercel (web), Render (API, render.yaml), Supabase (DB)	Deploy on every git push

The analysis pipeline



Invalid AI output is retried once, then rejected loudly. Failed quotes are kept, labelled Unverified, and can never enter the handover as fact. Results are cached by prompt hash, so re-running costs no quota and gives the same answer.

How the pieces fit together

- **Full run vs update run.** Each evidence unit records when it was analysed. A full run (first analysis or Start over) reads everything; an update run reads only unanalysed units, dedupes new claims against existing ones, reports only conflicts that involve a new claim, and flags approved claims those conflicts touch.
- **Out-of-date detection.** Each handover version stores a fingerprint of every approved claim (id + wording, status, owner, rationale). When the current approved set differs, the reviewer is told to make the next version.
- **Diffs.** Version N+1 is compared with the last approved version section by section (added / updated / removed). The diff is filtered with the same access rule as the handover, so restricted lines are counted, not shown.
- **Accounts on top of an existing model.** People the reviewer names start as placeholder users. Accepting an invite moves their membership, project role, acknowledgements and questions to the real account and marks the placeholder merged, so history stays intact and previews end.
- **Ask Baton.** Keyword-overlap plus fuzzy ranking over permitted units picks the top 12; the AI answers with citations; each citation must refer to a unit it was shown and pass Gate 1.
- **Upgrades in place.** On start-up the API adds any missing columns and marks evidence in already-analysed projects as analysed, so a Round 2 database became a Round 3 one with no data loss (tested on Postgres 16).
- **Data model.** Account, User, Invite, Project, Membership, Source, EvidenceUnit, Claim, ClaimEvidence, Gap, Conflict, HandoverVersion (snapshot + diff), Acknowledgement, Review, Question, AuditEvent, Job, LLMCache.

6. Reliability under normal use

What a judge might do	What Baton does
Refresh any page, or come back on another device	Sessions are signed tokens; accounts work from any device; every page reloads its data
Click the same button twice	Buttons lock while working; the server refuses repeats (resolve, dismiss, approve, accept invite) with a clear message
Wrong password, bad email, short password, duplicate account	Clear messages; sign-in throttled after repeated failures
Open an old, cancelled or used invite link	The invite page says exactly which, and what to do
Open a project they're not in	“Project not found, or you don't have access” and a way back — no existence leak
Analyse when nothing is new	“No new evidence since the last analysis” — no empty run
Upload an unsupported, empty or scanned file	Rejected or reported as unreadable with a reason; other files still go through
Gemini busy (503) or over quota (429)	One retry, then the backup model, 90 s timeout; otherwise a clear message and nothing half-saved
AI returns malformed JSON or invented quotes	Validated and retried once, then rejected; invented quotes never become facts or answers
Server restarts mid-analysis	The job is marked failed with “Re-run” instructions
Free hosting asleep	The home page says the server is waking up; an uptime ping keeps it awake

7. Testing details

Layer	What	Result
Automated (pytest)	23 end-to-end API tests with a scripted AI that makes deliberate mistakes (invented quote, one changed word, missing evidence, unsupported claim)	**23 / 23 pass** on SQLite with Python 3.12, 3.13, 3.14, 3.15 and on PostgreSQL 16
Round 3 tests	Accounts and bad input · guest upgrade · invites (create, replace, accept once, revoke, preview, remove) · worked-example switching · living handover (update run, decisions kept, flags, version 2 diff, ack carry-over) · GitHub re-sync · Ask Baton (checked quotes, refusal, access) · routing to the leaver · repeated actions and unknown IDs	9 / 9 pass
Browser (Playwright)	First-time reviewer journey; invite accepted in a second browser; preview; Ask Baton; living handover to version 2; refresh; worked example; error states; phone width	All pass, zero console errors
Upgrade	Round 2 Postgres database upgraded to the Round 3 schema at start-up	Pass, no data loss
Real services	Gemini schemas and the worked example with real Gemini on the team's PC; deployed site in incognito	Pass

Full test record: TESTING.md in the repository (scenario → test → result).

8. Usability and real-world fit

- **No walkthrough needed.** A pop-up guide points at the next button for each role and stage (add files → analyse → settle conflicts → review → build → approve → invite → keep it up to date). It hides while you type and can be turned off.
- **Plain words.** “Changed since approval”, “Not in the files”, “Ask Riya instead”, “Quote found word for word”.
- **Empty and error states** always say what to do next and offer a way back.
- **Works on a phone** for reading the handover and asking questions; reviewing is easiest on a laptop.
- **Real fit.** Built from our own problem: semester projects and college clubs lose knowledge every year when members rotate. The same flow fits small software teams, labs and offboarding in companies (see the roadmap).

9. Product roadmap and future development

When	What	Why
Next 3 months	Email delivery for invites and change notices · nightly GitHub checks · OCR for scans · Google Drive and Notion connectors · password reset · a labelled evaluation set measuring claim precision/recall per model	Make it dependable for real teams and measure quality instead of asserting it
3–6 months	Hybrid retrieval (keywords + pgvector embeddings) for Ask Baton, still quote-checked · Slack and Teams connectors · handover templates per team type · reviewer bulk tools · ownership map	Scale to larger projects without trusting the model more
6–12 months	Organisations and SSO · automatic handovers when someone is marked as leaving · private deployment with open-weight models · analytics (time-to-productive, stale handovers)	From a tool to a platform for colleges and companies

Who it's for: student teams and clubs first (free), then small software teams and agencies (per project), then departments, labs and companies (organisation plan with SSO). Principles that won't change: the AI proposes, code

checks every quote, people approve; missing knowledge is shown as a question; access is checked on every read; every change is logged. Full plan: ROADMAP.md.

10. Known limits, team and disclosures

Known limits (stated so nobody is surprised)

- Baton does not send email itself: invite links are copied or sent from the reviewer's mail app.
- Analysing your own files and Ask Baton call Gemini live; if the free tier is busy, Baton says so and a retry a minute later works. The worked example's analysis uses saved answers.
- Scanned (image-only) PDFs aren't read yet (no OCR); they are reported as unreadable. Max 10 MB per file.
- “Start over” re-reads everything and clears review decisions (it asks first); normal updates never do.
- The free API server sleeps after ~15 minutes idle; the first visit can take up to a minute.

Team

Member	Role	Work on the product
Shravani Yalagatti	Team Lead — AI & System Architecture	Designed and built Baton end to end in Rounds 2 and 3: architecture, data model, ingestion, AI extraction and verification, conflict/timeline/gap detection, accounts and invites, living handover, Ask Baton, web app, tests, deployment and documentation
Yash Solanke	Backend & Data Engineer (Round 1 plan)	Team member
Atharva Shete	Frontend, UX & Testing (Round 1 plan)	Team member

Disclosures

- **Built inside the event windows.** All product code was written during the official build windows; the Git commit history in the repository shows every step. No code from before the event was reused.
- **AI tools.** We used an AI coding assistant (Claude) to help write code and documentation. Inside the product, Google Gemini proposes claims and drafts answers, which code verifies and people approve.
- **Open-source libraries and APIs** under their licences: React, Vite, Tailwind, FastAPI, Pydantic, SQLAlchemy, psycopg, PyMuPDF, python-docx, pandas, RapidFuzz, PyJWT, httpx, google-genai, Playwright, pytest, and the GitHub REST API.
- **Data.** The worked example and sample project are entirely synthetic. Uploaded files stay in the project; secrets and phone numbers are redacted on import; passwords and invite tokens are stored only as hashes.